SBA301 – LAB 07

BUILDING DESKTOP APPLICATION USING JAVAFX AND HIBERNATE

HƯỚNG DẪN THỰC HÀNH CHI TIẾT STEP BY STEP

JavaFX • FXML • Scene Builder • Hibernate ORM • Repository Pattern • SQL Server

Case study: Library Department – Student / Book Management
Course baseline: JDK 21 • Maven • SQL Server • IntelliJ IDEA • Gluon Scene Builder

CORE PRINCIPLE: Mỗi bước phải tự kiểm chứng. "UI hiện lên" chưa đủ; sinh viên phải đối chiếu JavaFX state, Controller/Service call, Hibernate SQL và dữ liệu trong LibraryDB. Khi một checkpoint chưa PASS, không chuyển sang bước sau.

1. TÓM TẮT NỘI DUNG VÀ YÊU CẦU

Lab 07 xây dựng ứng dụng desktop cho Library Department để quản lý Student và Book. Theo tài liệu gốc, dữ liệu được lưu trong SQL Server thông qua Hibernate/JPA; ứng dụng sử dụng kiến trúc nhiều lớp với DAO, Repository, Service và JavaFX/FXML ở presentation layer. Quan hệ Student–Book được lưu qua bảng STUDENTS_BOOKS.

1.1. Kiến thức nền cần hiểu trước khi code

- ORM: ánh xạ Java object/entity sang bảng/row trong SQL Server.
- Hibernate SessionFactory là factory/runtime dùng lâu dài; Session là unit-of-work ngắn hạn.
- Transaction bao bọc thay đổi dữ liệu; commit/rollback quyết định trạng thái cuối trong DB.
- DAO chứa thao tác Hibernate trực tiếp; Repository che chi tiết DAO; Service cung cấp use case cho UI.
- JavaFX dùng Stage/Scene/Node; FXML mô tả UI; Controller xử lý event và gọi Service.
- TableView hiển thị danh sách; selected row cung cấp StudentID cho Update/Delete.
- Many-to-Many Student–Book dùng join table STUDENTS_BOOKS.

1.2. Luồng hệ thống cần đạt

```
JavaFX/FXML View
   ↓ event / selection
StudentController
   ↓
IStudentService / StudentService
   ↓
IStudentRepository / StudentRepository
   ↓
StudentDAO
   ↓
Hibernate Session / Transaction / HQL
   ↓
SQL Server: LibraryDB
   ├─ STUDENTS
   ├─ BOOKS
   └─ STUDENTS_BOOKS
```

1.3. Kết quả tối thiểu phải đạt

- LibraryDB tồn tại và kết nối được.
- Hibernate bootstrap thành công và tạo/validate mapping Student, Book, STUDENTS_BOOKS.
- StudentDAO thực hiện được Create, Read, Update, Delete.
- Repository và Service hoạt động đúng dependency direction.
- JavaFX Student Management page load được bằng FXML.
- TableView hiển thị Student từ DB.
- Click row đưa dữ liệu lên form và lấy đúng StudentID.
- Create/Update/Delete từ UI làm thay đổi DB đúng.

- Login page load được và login flow chuyển sang Student Management khi thành công.
- Có evidence: screenshot UI, SQL query, Hibernate SQL log và checklist PASS/FAIL.

2. CÔNG NGHỆ, PHẦN MỀM VÀ ĐIỀU KIỆN CHUẨN BỊ

Thành phần	Yêu cầu	Cách kiểm tra
JDK	21	java -version
Maven	3.6.3+ hoặc wrapper	mvn -v
IntelliJ IDEA	Community/Ultimate phù hợp course	IDE mở project
SQL Server	Instance truy cập được	SSMS SELECT 1
Gluon Scene Builder	Cài và mở được	Open/New FXML
JavaFX	Dependency/plugin đúng project	mvn javafx:run
Git	Khuyến nghị	git status

```
java -version
mvn -v
git --version

-- SQL Server smoke test
SELECT 1 AS ok;
```

CHECKPOINT: Nếu JDK/Maven/SQL Server/Scene Builder chưa hoạt động, sửa môi trường trước. Không debug Hibernate hoặc FXML khi toolchain chưa PASS.

3. CẤU TRÚC THƯ MỤC MỤC TIÊU

```
lab07-library/
├─ pom.xml
├─ src/main/java/fu/se/lab07/
│  ├─ entity/
│  │  ├─ Student.java
│  │  └─ Book.java
│  ├─ dao/
│  │  └─ StudentDAO.java
│  ├─ repository/
│  │  ├─ IStudentRepository.java
│  │  └─ StudentRepository.java
│  ├─ service/
│  │  ├─ IStudentService.java
│  │  └─ StudentService.java
│  ├─ util/
│  │  └─ HibernateUtil.java
│  └─ app/
│     └─ ConsoleSmokeTest.java
├─ src/main/resources/
│  └─ hibernate.cfg.xml
└─ JavaFX client/module
   ├─ controller/StudentController.java
   ├─ controller/LoginController.java
   ├─ StudentApplication.java
   ├─ student-view.fxml
   ├─ LoginGUI.fxml
   └─ module-info.java
```

ARCHITECTURE: Presentation layer không được gọi Hibernate Session trực tiếp. Controller chỉ gọi Service; DAO là lớp duy nhất biết rõ Session/HQL trong course baseline.

4. TÓM TẮT TOÀN BỘ CÁC BƯỚC THỰC HIỆN

Activity	Nội dung	Checkpoint chính
0	Chuẩn bị LibraryDB, JDK/Maven, Scene Builder	Toolchain + DB PASS
1	Design Student Management & Login	FXML mockup rõ fx:id/event
2	Hibernate architecture + DAO/Repository/Service	CRUD chạy không cần UI
3	JavaFX Student List + Controller CRUD	Full UI CRUD end-to-end
4	Login page	Login success -> Student Management

Activity	Nội dung	Checkpoint chính
5	Debug/Break-It/Test matrix	Evidence + G1–G16

5. ACTIVITY 0 – CHUẨN BỊ LIBRARYDB, TOOLCHAIN VÀ BASELINE

Step 0.1 – Tạo database LibraryDB

MỤC TIÊU: Tạo database riêng đúng tên Lab 07.

KIẾN THỨC NỀN CẦN NHỚ: Tên database là một phần của JDBC URL. Nếu sai databaseName, Hibernate fail ở bootstrap trước khi chạy CRUD.

THAO TÁC:

- Mở SSMS và kết nối đúng SQL Server instance.
- Mở New Query.
- Chạy script.

CODE / CẤU HÌNH:

```
IF DB_ID(N'LibraryDB') IS NULL
    CREATE DATABASE LibraryDB;
GO
USE LibraryDB;
GO
SELECT DB_NAME() AS current_db;
```

CÁCH CHẠY:

Execute trong SSMS.

CÁCH KIỂM TRA ĐÃ HOÀN THÀNH:

Query cuối phải trả LibraryDB.

PASS / FAIL: PASS khi LibraryDB tồn tại và query được. FAIL nếu permission/login/instance lỗi.

LỖI THƯỜNG GẶP / CÁCH XỬ LÝ:

- Nếu dùng named instance hoặc port khác 1433, ghi lại để cấu hình hibernate.cfg.xml.

Step 0.2 – Kiểm tra JDK/Maven/Git

MỤC TIÊU: Loại bỏ lỗi môi trường trước khi code.

KIẾN THỨC NỀN CẦN NHỚ: JDK dùng bởi Maven và JDK dùng bởi IntelliJ phải nhất quán; Lab khóa JDK 21.

CODE / CẤU HÌNH:

```
java -version
mvn -v
git status
```

CÁCH CHẠY:

Chạy tại CMD/PowerShell.

CÁCH KIỂM TRA ĐÃ HOÀN THÀNH:

Java 21; Maven chạy; Git không bắt buộc nhưng nên có checkpoint.

PASS / FAIL: PASS nếu toolchain đúng. FAIL: sửa JAVA_HOME/PATH hoặc Project SDK trước.

LỖI THƯỜNG GẶP / CÁCH XỬ LÝ:

- Maven có thể chạy bằng JDK khác IntelliJ; kiểm JAVA_HOME và mvn -v.

Step 0.3 – Kiểm tra Scene Builder

MỤC TIÊU: Đảm bảo sinh viên có thể thiết kế/mở FXML trước Activity 1.

KIẾN THỨC NỀN CẦN NHỚ: Scene Builder là visual editor cho FXML; FXML vẫn là source cần hiểu, không phải ảnh thiết kế tĩnh.

THAO TÁC:

- Mở Gluon Scene Builder.
- New -> Empty hoặc mở một FXML mẫu.
- Kéo thử TextField và Button.

CÁCH KIỂM TRA ĐÃ HOÀN THÀNH:

Scene Builder mở được và có thể Save FXML.

PASS / FAIL: PASS nếu file FXML được lưu và mở lại bình thường.

LỖI THƯỜNG GẶP / CÁCH XỬ LÝ:

- Nếu IntelliJ không Open in Scene Builder, cấu hình path Scene Builder trong Settings/Languages & Frameworks/JavaFX nếu IDE hỗ trợ.

6. ACTIVITY 1 – DESIGN THE STUDENT MANAGEMENT

Step 1.1 – Phân tích UI Manage Student

MỤC TIÊU: Xác định field, button, TableView và event trước khi viết controller.

KIẾN THỨC NỀN CẦN NHỚ: UI phải phản ánh use case CRUD. Mỗi control cần purpose và fx:id rõ để tránh FXML/Controller mismatch.

THAO TÁC:

- Tạo mockup Manage Student theo Lab gốc.
- Chuẩn bị các field: ID (read-only), Email, Password, First Name, Last Name, Marks.
- Chuẩn bị buttons: Add/Create, Update, Delete, Clear/Reset.
- Chuẩn bị TableView với các cột tương ứng.

CÁCH KIỂM TRA ĐÃ HOÀN THÀNH:

Có sơ đồ hoặc FXML mockup thể hiện form + action buttons + table.

PASS / FAIL: PASS khi mỗi control có name/fx:id dự kiến và không thiếu CRUD action.

LỖI THƯỜNG GẶP / CÁCH XỬ LÝ:

- Không dùng password TextField nếu muốn che input; PasswordField phù hợp hơn trong UI thực tế.

Step 1.2 – Phân tích UI Login

MỤC TIÊU: Thiết kế màn hình login tối thiểu trước khi code Activity 4.

KIẾN THỨC NỀN CẦN NHỚ: Login page chỉ thu thập credential và gọi service/validation; không truy cập DB trực tiếp từ Controller.

THAO TÁC:

- Tạo Email TextField.
- Tạo PasswordField.
- Tạo Login và Cancel/Exit button.

CÁCH KIỂM TRA ĐÃ HOÀN THÀNH:

Mockup Login có đủ email/password/login/cancel.

PASS / FAIL: PASS nếu Login UI có thể map rõ sang LoginController fields/event handlers.

7. ACTIVITY 2 – HIBERNATE ARCHITECTURE WITH REPOSITORY PATTERN

Step 2.1 – Tạo Maven project

MỤC TIÊU: Tạo core project chứa entity, DAO, Repository, Service và Hibernate config.

KIẾN THỨC NỀN CẦN NHỚ: Core persistence nên chạy độc lập trước JavaFX để giảm phạm vi debug.

THAO TÁC:

- IntelliJ -> File -> New -> Project -> Java/Maven.
- JDK 21.
- Artifact gợi ý: lab07-hibernate-core.

CÁCH KIỂM TRA ĐÃ HOÀN THÀNH:

pom.xml tồn tại; Maven sync xong.

PASS / FAIL: PASS nếu `mvn -q -DskipTests compile` thành công.

Step 2.2 – Khai báo dependencies

MỤC TIÊU: Thêm Hibernate ORM, Jakarta Persistence và SQL Server JDBC theo course baseline.

KIẾN THỨC NỀN CẦN NHỚ: Hibernate ORM thực hiện mapping/runtime; JDBC driver cung cấp kết nối SQL Server.

CODE / CẤU HÌNH:

```
<dependencies>
  <dependency>
    <groupId>org.hibernate.orm</groupId>
    <artifactId>hibernate-core</artifactId>
    <version>${hibernate.version}</version>
  </dependency>

  <dependency>
    <groupId>com.microsoft.sqlserver</groupId>
    <artifactId>mssql-jdbc</artifactId>
    <version>${mssql.version}</version>
  </dependency>

  <dependency>
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter</artifactId>
    <scope>test</scope>
  </dependency>
</dependencies>
```

CÁCH CHẠY:

Reload Maven, sau đó chạy `mvn dependency:tree`.

CÁCH KIỂM TRA ĐÃ HOÀN THÀNH:

Dependency tree có hibernate-core và mssql-jdbc.

PASS / FAIL: PASS khi compile sạch. FAIL nếu version conflict/missing artifact.

LỖI THƯỜNG GẶP / CÁCH XỬ LÝ:

- Dùng version phù hợp Lab/course; không tự nâng version chỉ vì mới hơn.

Step 2.3 – Tạo package structure

MỤC TIÊU: Tách entity/dao/repository/service/util để thể hiện Repository Architecture.

KIẾN THỨC NỀN CẦN NHỚ: Layering giúp UI không phụ thuộc Hibernate API.

THAO TÁC:

- Tạo packages: entity, dao, repository, service, util, app.

CÁCH KIỂM TRA ĐÃ HOÀN THÀNH:

Project tree khớp mục 3.

PASS / FAIL: PASS nếu mỗi package có responsibility rõ.

Step 2.4 – Tạo hibernate.cfg.xml

MỤC TIÊU: Cấu hình JDBC, dialect, DDL và mapped entities.

KIẾN THỨC NỀN CẦN NHỚ: Hibernate bootstrap đọc file cấu hình để build SessionFactory. `show_sql` giúp quan sát SQL; `hbm2ddl.auto=update` chỉ phù hợp lab/dev.

CODE / CẤU HÌNH:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE hibernate-configuration PUBLIC
        "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
        "https://www.hibernate.org/dtd/hibernate-configuration-3.0.dtd">
<hibernate-configuration>
  <session-factory>
    <property name="hibernate.connection.driver_class">com.microsoft.sqlserver.jdbc.SQLServerDriver</property>
    <property
name="hibernate.connection.url">jdbc:sqlserver://localhost:1433;databaseName=LibraryDB;encrypt=true;trustServerCer
tificate=true</property>
    <property name="hibernate.connection.username">sa</property>
    <property name="hibernate.connection.password">YOUR_PASSWORD</property>
    <property name="hibernate.dialect">org.hibernate.dialect.SQLServerDialect</property>
    <property name="hibernate.show_sql">true</property>
    <property name="hibernate.format_sql">true</property>
    <property name="hibernate.hbm2ddl.auto">update</property>
    <mapping class="fu.se.lab07.entity.Student"/>
    <mapping class="fu.se.lab07.entity.Book"/>
  </session-factory>
</hibernate-configuration>
```

CÁCH CHẠY:

Đặt file trong src/main/resources.

CÁCH KIỂM TRA ĐÃ HOÀN THÀNH:

File nằm đúng classpath; XML well-formed.

PASS / FAIL: PASS khi SessionFactory build ở bước 2.7.

LỖI THƯỜNG GẶP / CÁCH XỬ LÝ:

- Không commit password thật trong project dùng chung; lab có thể dùng placeholder/local config.

Step 2.5 – Tạo Book entity

MỤC TIÊU: Map bảng BOOKS theo mô tả Lab gốc.

KIẾN THỨC NỀN CẦN NHỚ: Entity cần @Entity, @Table, @Id và mapping column. Book là phía inverse trong Student–Book many-to-many nếu Student làm owner.

CODE / CẤU HÌNH:

```
@Entity
@Table(name = "BOOKS")
public class Book {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer id;

    @Column(nullable = false)
    private String author;

    @Column(nullable = false, unique = true)
    private String isbn;

    @Column(nullable = false)
    private String title;

    @ManyToMany(mappedBy = "books", fetch = FetchType.LAZY)
    private Set<Student> students = new HashSet<>();

    public Book() {}
    // getters/setters
}
```

CÁCH KIỂM TRA ĐÃ HOÀN THÀNH:

Class compile; annotations dùng jakarta.persistence.

PASS / FAIL: PASS khi compile và Hibernate bootstrap không báo unmapped class.

Step 2.6 – Tạo Student entity

MỤC TIÊU: Map bảng STUDENTS và relationship STUDENTS_BOOKS.

KIẾN THỨC NỀN CẦN NHỚ: Many-to-many dùng @JoinTable. Student làm owning side khi khai báo @JoinTable. Helper add/remove giữ object graph đồng bộ.

CODE / CẤU HÌNH:

```
@Entity
@Table(name = "STUDENTS")
public class Student {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer id;

    @Column(nullable = false, unique = true)
    private String email;

    @Column(nullable = false)
    private String password;

    @Column(nullable = false)
    private String firstName;

    @Column(nullable = false)
    private String lastName;

    private double marks;

    @ManyToMany(fetch = FetchType.LAZY)
    @JoinTable(
      name = "STUDENTS_BOOKS",
      joinColumns = @JoinColumn(name = "student_id"),
      inverseJoinColumns = @JoinColumn(name = "book_id")
    )
    private Set<Book> books = new HashSet<>();

    public Student() {}

    public void addBook(Book book) {
        books.add(book);
        book.getStudents().add(this);
    }

    public void removeBook(Book book) {
        books.remove(book);
        book.getStudents().remove(this);
    }
    // constructors/getters/setters
}
```

CÁCH KIỂM TRA ĐÃ HOÀN THÀNH:

Compile; relationship property name `books` khớp `mappedBy="books"` ở Book.

PASS / FAIL: PASS khi không có mapping exception.

Step 2.7 – Tạo HibernateUtil / SessionFactory

MỤC TIÊU: Build một SessionFactory dùng chung cho application core.

KIẾN THỨC NỀN CẦN NHỚ: SessionFactory expensive và long-lived; Session short-lived. Không build factory cho từng CRUD.

CODE / CẤU HÌNH:

```
public final class HibernateUtil {
    private static final SessionFactory SESSION_FACTORY = build();

    private static SessionFactory build() {
        try {
            return new Configuration()
                .configure("hibernate.cfg.xml")
                .buildSessionFactory();
        } catch (Exception ex) {
            throw new ExceptionInInitializerError(ex);
        }
    }
```

```
    public static SessionFactory getSessionFactory() {
        return SESSION_FACTORY;
    }

    public static void shutdown() {
        SESSION_FACTORY.close();
    }

    private HibernateUtil() {}
}
```

CÁCH CHẠY:

Tạo một ConsoleSmokeTest gọi getSessionFactory().

CÁCH KIỂM TRA ĐÃ HOÀN THÀNH:

Console không exception; SQL Server thấy STUDENTS, BOOKS, STUDENTS_BOOKS sau bootstrap nếu DDL update.

PASS / FAIL: PASS khi factory build thành công và DB schema đúng.

LỖI THƯỜNG GẶP / CÁCH XỬ LÝ:

- Nếu login failed: kiểm URL/user/password; nếu invalid object/table: kiểm mapping/schema/dialect.

Step 2.8 – Kiểm tra schema trong SQL Server

MỤC TIÊU: Chứng minh mapping Hibernate tạo/nhận đúng ba bảng của Lab.

KIẾN THỨC NỀN CẦN NHỚ: DB state là evidence cuối cùng của persistence mapping, không chỉ console log.

CODE / CẤU HÌNH:

```
USE LibraryDB;
GO
SELECT TABLE_NAME
FROM INFORMATION_SCHEMA.TABLES
WHERE TABLE_NAME IN ('STUDENTS','BOOKS','STUDENTS_BOOKS');
```

CÁCH CHẠY:

Execute trong SSMS sau khi core app bootstrap.

CÁCH KIỂM TRA ĐÃ HOÀN THÀNH:

Có đủ 3 table.

PASS / FAIL: PASS khi đủ STUDENTS, BOOKS, STUDENTS_BOOKS.

Step 2.9 – Tạo StudentDAO skeleton

MỤC TIÊU: Cô lập Hibernate Session/Transaction trong DAO.

KIẾN THỨC NỀN CẦN NHỚ: DAO là persistence implementation. Mỗi write operation cần begin/commit/rollback/close.

CODE / CẤU HÌNH:

```
public class StudentDAO {
    private final SessionFactory sessionFactory;

    public StudentDAO(SessionFactory sessionFactory) {
        this.sessionFactory = sessionFactory;
    }
}
```

CÁCH KIỂM TRA ĐÃ HOÀN THÀNH:

StudentDAO không phụ thuộc JavaFX.

PASS / FAIL: PASS nếu DAO chỉ import Hibernate/entity/util concerns.

Step 2.10 – Implement CREATE trong StudentDAO

MỤC TIÊU: Persist Student mới và lấy generated ID.

KIẾN THỨC NỀN CẦN NHỚ: `persist()` chuyển transient entity thành managed. Commit/flush phát INSERT.

CODE / CẤU HÌNH:

```
public Student save(Student student) {
    try (Session session = sessionFactory.openSession()) {
        Transaction tx = session.beginTransaction();
        try {
            session.persist(student);
            tx.commit();
            return student;
        } catch (RuntimeException ex) {
            tx.rollback();
            throw ex;
        }
    }
}
```

CÁCH KIỂM TRA ĐÃ HOÀN THÀNH:

Sau gọi save, student.id khác null và DB có row.

PASS / FAIL: PASS khi Hibernate SQL có INSERT và SELECT DB thấy dữ liệu.

Step 2.11 – Implement READ ALL / FIND BY ID

MỤC TIÊU: Đọc danh sách và một Student theo id.

KIẾN THỨC NỀN CẦN NHỚ: HQL dùng entity/property names; `session.get()` phù hợp PK lookup.

CODE / CẤU HÌNH:

```
public List<Student> findAll() {
    try (Session session = sessionFactory.openSession()) {
        return session.createQuery(
            "from Student s order by s.id", Student.class
        ).getResultList();
    }
}

public Optional<Student> findById(Integer id) {
    try (Session session = sessionFactory.openSession()) {
        return Optional.ofNullable(session.get(Student.class, id));
    }
}
```

CÁCH KIỂM TRA ĐÃ HOÀN THÀNH:

findAll trả danh sách; missing ID trả Optional.empty.

PASS / FAIL: PASS khi SQL SELECT đúng và không throw Lazy issue ở dữ liệu cơ bản.

Step 2.12 – Implement UPDATE

MỤC TIÊU: Cập nhật Student nhận từ UI/detached object.

KIẾN THỨC NỀN CẦN NHỚ: `merge()` copy state vào managed instance; return value mới là managed reference.

CODE / CẤU HÌNH:

```
public Student update(Student detached) {
    try (Session session = sessionFactory.openSession()) {
        Transaction tx = session.beginTransaction();
        try {
            Student managed = session.merge(detached);
            tx.commit();
            return managed;
        } catch (RuntimeException ex) {
            tx.rollback();
            throw ex;
        }
    }
}
```

CÁCH KIỂM TRA ĐÃ HOÀN THÀNH:

DB row thay đổi sau update.

PASS / FAIL: PASS khi UPDATE SQL xuất hiện và GET lại phản ánh giá trị mới.

Step 2.13 – Implement DELETE

MỤC TIÊU: Xóa Student theo id.

KIẾN THỨC NỀN CẦN NHỚ: Phải phân biệt delete entity và remove association; DB FK/join table vẫn quyết định behavior.

CODE / CẤU HÌNH:

```
public boolean delete(Integer id) {
    try (Session session = sessionFactory.openSession()) {
        Transaction tx = session.beginTransaction();
        try {
            Student student = session.get(Student.class, id);
            if (student == null) {
                tx.rollback();
                return false;
            }
            session.remove(student);
            tx.commit();
            return true;
        } catch (RuntimeException ex) {
            if (tx.isActive()) tx.rollback();
            throw ex;
        }
    }
}
```

CÁCH KIỂM TRA ĐÃ HOÀN THÀNH:

GET lại id trả empty; SQL row không còn.

PASS / FAIL: PASS khi delete đúng entity và không vô tình xóa shared Book.

Step 2.14 – Tạo IStudentRepository

MỤC TIÊU: Định nghĩa persistence contract mà Service sử dụng.

KIẾN THỨC NỀN CẦN NHỚ: Repository che chi tiết DAO/Hibernate khỏi Service.

CODE / CẤU HÌNH:

```
public interface IStudentRepository {
    Student save(Student student);
    List<Student> findAll();
    Optional<Student> findById(Integer id);
    Student update(Student student);
    boolean delete(Integer id);
}
```

CÁCH KIỂM TRA ĐÃ HOÀN THÀNH:

Interface không import Hibernate Session/Transaction.

PASS / FAIL: PASS khi contract chỉ dùng domain types/collections.

Step 2.15 – Tạo StudentRepository

MỤC TIÊU: Implement Repository bằng StudentDAO.

KIẾN THỨC NỀN CẦN NHỚ: Course source dùng Repository Pattern để tách persistence contract khỏi implementation.

CODE / CẤU HÌNH:

```
public class StudentRepository implements IStudentRepository {
    private final StudentDAO dao;

    public StudentRepository(StudentDAO dao) { this.dao = dao; }

    public Student save(Student s) { return dao.save(s); }
    public List<Student> findAll() { return dao.findAll(); }
    public Optional<Student> findById(Integer id) { return dao.findById(id); }
    public Student update(Student s) { return dao.update(s); }
    public boolean delete(Integer id) { return dao.delete(id); }
}
```

CÁCH KIỂM TRA ĐÃ HOÀN THÀNH:

Repository compile và forward đúng methods.

PASS / FAIL: PASS khi Service có thể dùng IStudentRepository mà không biết DAO.

Step 2.16 – Tạo IStudentService và StudentService

MỤC TIÊU: Tạo use-case boundary cho JavaFX Controller.

KIẾN THỨC NỀN CẦN NHỚ: Service là nơi phù hợp cho validation/not-found/business rules; UI không gọi DAO.

CODE / CẤU HÌNH:

```
public interface IStudentService {
    Student create(Student student);
    List<Student> getAll();
    Student getById(Integer id);
    Student update(Student student);
    void delete(Integer id);
}

public class StudentService implements IStudentService {
    private final IStudentRepository repository;

    public StudentService(IStudentRepository repository) {
        this.repository = repository;
    }

    public Student create(Student s) { validate(s); return repository.save(s); }
    public List<Student> getAll() { return repository.findAll(); }
    public Student getById(Integer id) {
        return repository.findById(id)
            .orElseThrow(() -> new IllegalArgumentException("Student not found"));
    }
    public Student update(Student s) { validate(s); return repository.update(s); }
    public void delete(Integer id) {
        if (!repository.delete(id)) throw new IllegalArgumentException("Student not found");
    }

    private void validate(Student s) {
        if (s == null) throw new IllegalArgumentException("Student is required");
        if (s.getEmail() == null || s.getEmail().isBlank())
            throw new IllegalArgumentException("Email is required");
        if (s.getMarks() < 0 || s.getMarks() > 10)
            throw new IllegalArgumentException("Marks must be between 0 and 10");
    }
}
```

CÁCH KIỂM TRA ĐÃ HOÀN THÀNH:

Service compile; Controller sau này chỉ cần IStudentService.

PASS / FAIL: PASS nếu Service không import Hibernate API.

Step 2.17 – Core smoke test trước JavaFX

MỤC TIÊU: Chứng minh CRUD backend/core chạy độc lập.

KIẾN THỨC NỀN CẦN NHỚ: Nếu core chưa PASS thì lỗi ở JavaFX sau này không thể chẩn đoán chính xác.

CODE / CẤU HÌNH:

```
public class ConsoleSmokeTest {
    public static void main(String[] args) {
        SessionFactory sf = HibernateUtil.getSessionFactory();
        StudentDAO dao = new StudentDAO(sf);
        IStudentRepository repo = new StudentRepository(dao);
        IStudentService service = new StudentService(repo);

        Student s = new Student();
        s.setEmail("lab07@fpt.edu.vn");
        s.setPassword("123456");
        s.setFirstName("Lab");
        s.setLastName("Seven");
        s.setMarks(8.0);

        Student created = service.create(s);
        System.out.println("created id=" + created.getId());
        service.getAll().forEach(System.out::println);
        HibernateUtil.shutdown();
```

```
    }
}
```

CÁCH CHẠY:

Run ConsoleSmokeTest trong IntelliJ.

CÁCH KIỂM TRA ĐÃ HOÀN THÀNH:

Console có generated ID; DB có row Student.

PASS / FAIL: PASS khi CRUD core chạy mà không cần JavaFX. Đây là checkpoint bắt buộc.

8. ACTIVITY 3 – DEVELOP THE STUDENT LIST PAGE WITH JAVAFX/FXML

Step 3.1 – Tạo JavaFX Maven project/module

MỤC TIÊU: Tạo presentation project để hiển thị Student list và CRUD form.

KIẾN THỨC NỀN CẦN NHỚ: JavaFX client nên phụ thuộc Service/core, không copy DAO sang UI project.

THAO TÁC:

- IntelliJ -> New Project -> JavaFX hoặc Maven JavaFX.
- JDK 21.
- Artifact gợi ý: lab07-javafx-ui.

CÁCH KIỂM TRA ĐÃ HOÀN THÀNH:

Project JavaFX compile baseline.

PASS / FAIL: PASS khi một Hello JavaFX screen chạy được.

Step 3.2 – Khai báo JavaFX dependencies/plugin

MỤC TIÊU: Cho project compile/run controls và FXML.

KIẾN THỨC NỀN CẦN NHỚ: FXML cần javafx.fxml; controls cần javafx.controls.

CODE / CẤU HÌNH:

```
<dependency>
  <groupId>org.openjfx</groupId>
  <artifactId>javafx-controls</artifactId>
  <version>${javafx.version}</version>
</dependency>
<dependency>
  <groupId>org.openjfx</groupId>
  <artifactId>javafx-fxml</artifactId>
  <version>${javafx.version}</version>
</dependency>

<plugin>
  <groupId>org.openjfx</groupId>
  <artifactId>javafx-maven-plugin</artifactId>
  <version>${javafx.maven.plugin.version}</version>
  <configuration>
    <mainClass>fu.se.lab07.ui/fu.se.lab07.ui.StudentApplication</mainClass>
  </configuration>
</plugin>
```

CÁCH CHẠY:

Reload Maven.

CÁCH KIỂM TRA ĐÃ HOÀN THÀNH:

`mvn clean javafx:run` có thể launch app sau khi main class/FXML có sẵn.

PASS / FAIL: PASS khi dependency resolve sạch.

Step 3.3 – Cấu hình module-info.java

MỤC TIÊU: Cho JavaFX/FXML truy cập controller bằng module reflection.

KIẾN THỨC NỀN CẦN NHỚ: `exports` cho public access; `opens ... to javafx.fxml` cho reflective @FXML injection.

CODE / CẤU HÌNH:

```
module fu.se.lab07.ui {
    requires javafx.controls;
    requires javafx.fxml;

    // Nếu core là module khác:
    // requires fu.se.lab07.core;

    exports fu.se.lab07.ui;
    opens fu.se.lab07.ui.controller to javafx.fxml;
}
```

CÁCH KIỂM TRA ĐÃ HOÀN THÀNH:

Project compile; không InaccessibleObjectException khi load FXML.

PASS / FAIL: PASS khi FXMLLoader access Controller package.

LỖI THƯỜNG GẶP / CÁCH XỬ LÝ:

- Nếu project không dùng JPMS, module-info có thể bỏ; nhưng tài liệu Lab gốc có module-info nên guide giữ nội dung này.

Step 3.4 – Kết nối core project/service

MỤC TIÊU: Cho JavaFX UI dùng StudentService đã PASS ở Activity 2.

KIẾN THỨC NỀN CẦN NHỚ: Tránh copy source giữa projects. Dùng Maven module/dependency hoặc project reference theo course environment.

THAO TÁC:

- Nếu multi-module Maven: khai báo dependency vào core artifact.
- Nếu course dùng IntelliJ Project Structure: Add Module Dependency/Reference như Lab gốc.

CÁCH KIỂM TRA ĐÃ HOÀN THÀNH:

JavaFX project import được Student và IStudentService.

PASS / FAIL: PASS khi compile mà không duplicate entity/service source.

Step 3.5 – Tạo student-view.fxml bằng Scene Builder

MỤC TIÊU: Xây UI Manage Student theo thiết kế Activity 1.

KIẾN THỨC NỀN CẦN NHỚ: FXML cần fx:controller, fx:id chính xác; button onAction phải map đúng Controller method.

CODE / CẤU HÌNH:

```
<?xml version="1.0" encoding="UTF-8"?>
<?import javafx.scene.control.*?>
<?import javafx.scene.layout.*?>

<BorderPane xmlns="http://javafx.com/javafx"
            xmlns:fx="http://javafx.com/fxml"
            fx:controller="fu.se.lab07.ui.controller.StudentController">
  <top>
    <Label text="Manage Student"/>
  </top>
  <center>
    <VBox spacing="10">
      <GridPane hgap="10" vgap="8">
        <Label text="Email" GridPane.rowIndex="0"/>
        <TextField fx:id="txtEmail" GridPane.columnIndex="1" GridPane.rowIndex="0"/>
        <Label text="Password" GridPane.rowIndex="1"/>
        <PasswordField fx:id="txtPassword" GridPane.columnIndex="1" GridPane.rowIndex="1"/>
        <Label text="First Name" GridPane.rowIndex="2"/>
        <TextField fx:id="txtFirstName" GridPane.columnIndex="1" GridPane.rowIndex="2"/>
        <Label text="Last Name" GridPane.rowIndex="3"/>
        <TextField fx:id="txtLastName" GridPane.columnIndex="1" GridPane.rowIndex="3"/>
        <Label text="Marks" GridPane.rowIndex="4"/>
        <TextField fx:id="txtMarks" GridPane.columnIndex="1" GridPane.rowIndex="4"/>
      </GridPane>
      <HBox spacing="8">
        <Button text="Add Student" onAction="#handleCreate"/>
        <Button text="Update Student" onAction="#handleUpdate"/>
```

```
        <Button text="Delete Student" onAction="#handleDelete"/>
        <Button text="Clear" onAction="#handleClear"/>
      </HBox>
      <TableView fx:id="tblStudents" prefHeight="320">
        <columns>
          <TableColumn fx:id="colId" text="ID"/>
          <TableColumn fx:id="colEmail" text="Email"/>
          <TableColumn fx:id="colFirstName" text="First Name"/>
          <TableColumn fx:id="colLastName" text="Last Name"/>
          <TableColumn fx:id="colMarks" text="Marks"/>
        </columns>
      </TableView>
    </VBox>
  </center>
</BorderPane>
```

CÁCH KIỂM TRA ĐÃ HOÀN THÀNH:

Scene Builder preview đúng; FXML lưu trong resources.

PASS / FAIL: PASS nếu FXMLLoader load được ở bước 3.7.

Step 3.6 – Tạo StudentController fields và initialize()

MỤC TIÊU: Inject controls, cấu hình TableView và selection listener.

KIẾN THỨC NỀN CẦN NHỚ: Không access @FXML field trong constructor; injection xảy ra trước initialize().

CODE / CẤU HÌNH:

```
public class StudentController {
    @FXML private TextField txtEmail;
    @FXML private PasswordField txtPassword;
    @FXML private TextField txtFirstName;
    @FXML private TextField txtLastName;
    @FXML private TextField txtMarks;

    @FXML private TableView<Student> tblStudents;
    @FXML private TableColumn<Student,Integer> colId;
    @FXML private TableColumn<Student,String> colEmail;
    @FXML private TableColumn<Student,String> colFirstName;
    @FXML private TableColumn<Student,String> colLastName;
    @FXML private TableColumn<Student,Double> colMarks;

    private final ObservableList<Student> data = FXCollections.observableArrayList();
    private IStudentService studentService;
    private Integer selectedStudentId;

    @FXML
    private void initialize() {
        colId.setCellValueFactory(new PropertyValueFactory<>("id"));
        colEmail.setCellValueFactory(new PropertyValueFactory<>("email"));
        colFirstName.setCellValueFactory(new PropertyValueFactory<>("firstName"));
        colLastName.setCellValueFactory(new PropertyValueFactory<>("lastName"));
        colMarks.setCellValueFactory(new PropertyValueFactory<>("marks"));
        tblStudents.setItems(data);
        tblStudents.getSelectionModel().selectedItemProperty()
            .addListener((obs, oldV, selected) -> populateForm(selected));
    }
}
```

CÁCH KIỂM TRA ĐÃ HOÀN THÀNH:

FXML load không null field; TableColumns hiển thị header.

PASS / FAIL: PASS nếu initialize chạy sau injection.

LỖI THƯỜNG GẶP / CÁCH XỬ LÝ:

- PropertyValueFactory string phải khớp Java bean getter names.

Step 3.7 – Wire Service từ composition root

MỤC TIÊU: Inject Service vào Controller ngoài FXML lifecycle.

KIẾN THỨC NỀN CẦN NHỚ: Controller không được `new StudentDAO()`; wiring nằm ở Application/composition root.

CODE / CẤU HÌNH:

```
FXMLLoader loader = new FXMLLoader(
    StudentApplication.class.getResource("/fu/se/lab07/ui/student-view.fxml")
);
Parent root = loader.load();
StudentController controller = loader.getController();
controller.setStudentService(studentService);

stage.setScene(new Scene(root));
stage.show();
```

CÁCH KIỂM TRA ĐÃ HOÀN THÀNH:

Controller nhận service và có thể gọi refresh.

PASS / FAIL: PASS khi UI load và service không null.

Step 3.8 – Show all Students

MỤC TIÊU: Đổ dữ liệu DB vào TableView.

KIẾN THỨC NỀN CẦN NHỚ: ObservableList là UI model; DB source-of-truth vẫn qua Service.

CODE / CẤU HÌNH:

```
public void setStudentService(IStudentService service) {
    this.studentService = service;
    refreshTable();
}

private void refreshTable() {
    data.setAll(studentService.getAll());
}
```

CÁCH KIỂM TRA ĐÃ HOÀN THÀNH:

TableView số row khớp SELECT COUNT(*) hoặc danh sách trong DB.

PASS / FAIL: PASS khi Student list hiển thị đúng dữ liệu.

Step 3.9 – Lấy StudentID khi click row

MỤC TIÊU: Đồng bộ selected row với form và selectedStudentId.

KIẾN THỨC NỀN CẦN NHỚ: Update/Delete phải dựa identity rõ ràng; không suy luận ID từ vị trí row.

CODE / CẤU HÌNH:

```
private void populateForm(Student s) {
    if (s == null) return;
    selectedStudentId = s.getId();
    txtEmail.setText(s.getEmail());
    txtPassword.setText(s.getPassword());
    txtFirstName.setText(s.getFirstName());
    txtLastName.setText(s.getLastName());
    txtMarks.setText(Double.toString(s.getMarks()));
}
```

CÁCH KIỂM TRA ĐÃ HOÀN THÀNH:

Click row: form prefill và debugger cho thấy selectedStudentId đúng row.

PASS / FAIL: PASS khi selection luôn đồng bộ.

Step 3.10 – Parse/validate form

MỤC TIÊU: Chặn input sai trước khi gọi Service.

KIẾN THỨC NỀN CẦN NHỚ: UI validation xử lý format/required; Service vẫn kiểm business rules.

CODE / CẤU HÌNH:

```
private Student readForm() {
    String email = txtEmail.getText().trim();
    String password = txtPassword.getText();
    String first = txtFirstName.getText().trim();
    String last = txtLastName.getText().trim();

    if (email.isBlank() || password.isBlank() || first.isBlank() || last.isBlank())
        throw new IllegalArgumentException("All text fields are required");
```

```
    double marks;
    try {
        marks = Double.parseDouble(txtMarks.getText().trim());
    } catch (NumberFormatException ex) {
        throw new IllegalArgumentException("Marks must be numeric");
    }
    if (marks < 0 || marks > 10)
        throw new IllegalArgumentException("Marks must be between 0 and 10");

    Student s = new Student();
    s.setEmail(email);
    s.setPassword(password);
    s.setFirstName(first);
    s.setLastName(last);
    s.setMarks(marks);
    return s;
}
```

CÁCH KIỂM TRA ĐÃ HOÀN THÀNH:

Invalid marks không gọi Service.

PASS / FAIL: PASS nếu UI báo lỗi thay vì crash.

Step 3.11 – CREATE từ JavaFX

MỤC TIÊU: Tạo Student mới từ form.

KIẾN THỨC NỀN CẦN NHỚ: Handler nên mỏng: read/validate -> service -> refresh -> clear -> feedback.

CODE / CẤU HÌNH:

```
@FXML
private void handleCreate() {
    try {
        studentService.create(readForm());
        refreshTable();
        clearForm();
        showInfo("Student created");
    } catch (RuntimeException ex) {
        showError(ex.getMessage());
    }
}
```

CÁCH KIỂM TRA ĐÃ HOÀN THÀNH:

Click Add: row mới xuất hiện và SQL Server có INSERT.

PASS / FAIL: PASS khi UI + Hibernate log + DB cùng xác nhận.

Step 3.12 – UPDATE từ JavaFX

MỤC TIÊU: Cập nhật Student đang chọn.

KIẾN THỨC NỀN CẦN NHỚ: Phải giữ đúng ID của row được chọn; detached entity sẽ được Service/DAO merge theo course flow.

CODE / CẤU HÌNH:

```
@FXML
private void handleUpdate() {
    if (selectedStudentId == null) {
        showError("Select a student first");
        return;
    }
    try {
        Student s = readForm();
        s.setId(selectedStudentId);
        studentService.update(s);
        refreshTable();
        clearForm();
        showInfo("Student updated");
    } catch (RuntimeException ex) {
        showError(ex.getMessage());
    }
}
```

CÁCH KIỂM TRA ĐÃ HOÀN THÀNH:

Update selected row; DB value thay đổi đúng ID.

PASS / FAIL: PASS khi không update nhầm Student.

Step 3.13 – DELETE từ JavaFX

MỤC TIÊU: Xóa Student được chọn với confirmation.

KIẾN THỨC NỀN CẦN NHỚ: UI confirmation là UX; business/persistence correctness vẫn nằm Service/DAO/DB.

CODE / CẤU HÌNH:

```
@FXML
private void handleDelete() {
    if (selectedStudentId == null) {
        showError("Select a student first");
        return;
    }
    if (!confirm("Delete selected student?")) return;
    try {
        studentService.delete(selectedStudentId);
        refreshTable();
        clearForm();
        showInfo("Student deleted");
    } catch (RuntimeException ex) {
        showError(ex.getMessage());
    }
}
```

CÁCH KIỂM TRA ĐÃ HOÀN THÀNH:

Row biến mất; SQL DELETE xuất hiện; DB không còn record.

PASS / FAIL: PASS khi delete đúng selectedStudentId.

Step 3.14 – Clear form và Alert helpers

MỤC TIÊU: Giữ UI state nhất quán sau CRUD.

KIẾN THỨC NỀN CẦN NHỚ: Sau mutation phải reset selected ID/selection để tránh update nhầm row.

CODE / CẤU HÌNH:

```
@FXML
private void handleClear() { clearForm(); }

private void clearForm() {
    selectedStudentId = null;
    txtEmail.clear(); txtPassword.clear();
    txtFirstName.clear(); txtLastName.clear(); txtMarks.clear();
    tblStudents.getSelectionModel().clearSelection();
    txtEmail.requestFocus();
}

private void showInfo(String message) {
    new Alert(Alert.AlertType.INFORMATION, message, ButtonType.OK).showAndWait();
}
private void showError(String message) {
    new Alert(Alert.AlertType.ERROR, message, ButtonType.OK).showAndWait();
}
private boolean confirm(String message) {
    return new Alert(Alert.AlertType.CONFIRMATION, message, ButtonType.YES, ButtonType.NO)
        .showAndWait().orElse(ButtonType.NO) == ButtonType.YES;
}
```

CÁCH KIỂM TRA ĐÃ HOÀN THÀNH:

Clear xóa form + selection; Alert hiển thị đúng.

PASS / FAIL: PASS khi UI không giữ selectedStudentId cũ sau Create/Delete.

Step 3.15 – Chạy full Student Management

MỤC TIÊU: Kiểm toàn bộ UI CRUD end-to-end.

KIẾN THỨC NỀN CẦN NHỚ: Một operation chỉ PASS khi UI, generated SQL và DB state nhất quán.

CÁCH CHẠY:

`mvn clean javafx:run` hoặc Run StudentApplication trong IntelliJ.

CÁCH KIỂM TRA ĐÃ HOÀN THÀNH:

Thực hiện create -> select -> update -> delete; đối chiếu DB sau mỗi operation.

PASS / FAIL: PASS khi 4 CRUD đều hoạt động và không layer violation.

9. ACTIVITY 4 – DESIGN AND DEVELOP THE LOGIN PAGE

Step 4.1 – Tạo LoginGUI.fxml

MỤC TIÊU: Tạo màn hình Login theo Lab gốc.

KIẾN THỨC NỀN CẦN NHỚ: FXML declarative view; Controller xử lý credential/event.

CODE / CẤU HÌNH:

```
<VBox xmlns="http://javafx.com/javafx"
      xmlns:fx="http://javafx.com/fxml"
      fx:controller="fu.se.lab07.ui.controller.LoginController"
      alignment="CENTER" spacing="10">
  <Label text="LOGIN FORM"/>
  <TextField fx:id="txtEmail" promptText="Email"/>
  <PasswordField fx:id="txtPassword" promptText="Password"/>
  <HBox spacing="8" alignment="CENTER">
    <Button text="Login" onAction="#handleLogin"/>
    <Button text="Cancel" onAction="#handleCancel"/>
  </HBox>
</VBox>
```

CÁCH KIỂM TRA ĐÃ HOÀN THÀNH:

Scene Builder preview và FXML load được.

PASS / FAIL: PASS khi fx:id/handler/controller names khớp.

Step 4.2 – Bổ sung login lookup vào Repository/Service

MỤC TIÊU: Có use case xác minh email/password cho màn hình Login.

KIẾN THỨC NỀN CẦN NHỚ: Lab gốc yêu cầu Login page nhưng không mô tả security framework; trong phạm vi Lab 07, login có thể là kiểm credential từ Student data theo course baseline.

CODE / CẤU HÌNH:

```
// DAO
public Optional<Student> findByEmail(String email) {
    try (Session session = sessionFactory.openSession()) {
        return session.createQuery(
            "from Student s where s.email = :email", Student.class
        ).setParameter("email", email)
         .uniqueResultOptional();
    }
}

// Service
public Student login(String email, String password) {
    Student s = repository.findByEmail(email)
        .orElseThrow(() -> new IllegalArgumentException("Invalid email or password"));
    if (!s.getPassword().equals(password))
        throw new IllegalArgumentException("Invalid email or password");
    return s;
}
```

CÁCH KIỂM TRA ĐÃ HOÀN THÀNH:

Login valid trả Student; invalid email/password bị từ chối.

PASS / FAIL: PASS cho mục tiêu Lab 07. WARNING: plaintext password comparison chỉ là course baseline theo schema/source; không dùng production.

LỖI THƯỜNG GẶP / CÁCH XỬ LÝ:

- Phần Advanced sẽ yêu cầu hash password/refactor auth.

Step 4.3 – Tạo LoginController

MỤC TIÊU: Đọc credential, gọi Service và chuyển sang Student Management khi login thành công.

KIẾN THỨC NỀN CẦN NHỚ: Controller chỉ điều phối UI; DB query nằm Repository/DAO.

CODE / CẤU HÌNH:

```
public class LoginController {
    @FXML private TextField txtEmail;
    @FXML private PasswordField txtPassword;

    private IStudentService studentService;

    public void setStudentService(IStudentService service) {
        this.studentService = service;
    }

    @FXML
    private void handleLogin() {
        try {
            Student user = studentService.login(
                txtEmail.getText().trim(), txtPassword.getText());
            openStudentManagement(user);
        } catch (RuntimeException ex) {
            new Alert(Alert.AlertType.ERROR, ex.getMessage()).showAndWait();
        }
    }

    @FXML
    private void handleCancel() {
        ((Stage) txtEmail.getScene().getWindow()).close();
    }
}
```

CÁCH KIỂM TRA ĐÃ HOÀN THÀNH:

Invalid credential không mở Student Management; valid credential mở screen tiếp theo.

PASS / FAIL: PASS khi flow đúng và LoginController không import Hibernate.

Step 4.4 – Chuyển scene sau Login Success

MỤC TIÊU: Hoàn thiện flow Login -> Manage Student như trang cuối Lab gốc.

KIẾN THỨC NỀN CẦN NHỚ: Khi navigation sang FXML mới, vẫn phải inject cùng Service graph; không tạo DAO/SessionFactory mới trong controller mới.

CODE / CẤU HÌNH:

```
private void openStudentManagement(Student loggedIn) throws IOException {
    FXMLLoader loader = new FXMLLoader(
        getClass().getResource("/fu/se/lab07/ui/student-view.fxml")
    );
    Parent root = loader.load();
    StudentController controller = loader.getController();
    controller.setStudentService(studentService);

    Stage stage = (Stage) txtEmail.getScene().getWindow();
    stage.setScene(new Scene(root));
    stage.setTitle("Manage Student");
    stage.show();
}
```

CÁCH KIỂM TRA ĐÃ HOÀN THÀNH:

Login success -> Student Management; list load được.

PASS / FAIL: PASS khi navigation không tạo duplicate persistence infrastructure.

10. DEBUGGING VÀ BREAK-IT → DIAGNOSE → FIX

Break	Triệu chứng	Nguyên nhân cần kiểm	Fix
Sai databaseName	SessionFactory bootstrap fail	JDBC URL/DB name	Sửa hibernate.cfg.xml
Sai mappedBy	Mapping exception	Tên Java property	mappedBy="books"
Không commit transaction	DB không đổi	tx lifecycle	commit/rollback đúng

Break	Triệu chứng	Nguyên nhân cần kiểm	Fix
Session singleton	stale/thread/resource lỗi	Session scope	short-lived Session
fx:id mismatch	@FXML field null/NPE	FXML vs Controller	đồng bộ tên
Sai onAction	FXMLLoadException	handler method	sửa #handle...
Thiếu opens	InaccessibleObjectException	module reflection	opens controller to javafx.fxml
Marks = abc	NumberFormatException	UI validation	parse + error
Update khi chưa select	sai/null ID	selection state	guard selectedStudentId
UI gọi DAO trực tiếp	architecture fail	dependency review	Controller -> Service
Login query trong Controller	coupling/test khó	layer violation	move xuống Service/Repository

11. TEST MATRIX BẮT BUỘC

ID	Test	Expected evidence
T01	SessionFactory bootstrap	No exception + tables exist
T02	Create Student core	INSERT + generated ID
T03	Find all	SELECT + correct list
T04	Find missing ID	Optional.empty / not-found
T05	Update Student	UPDATE + DB changed
T06	Delete Student	DELETE + row gone
T07	FXML load	No FXMLLoader exception
T08	Initial TableView	Rows match DB
T09	Row selection	Form + selectedStudentId sync
T10	Create from UI	UI + SQL INSERT + DB row
T11	Update from UI	selected ID updated only
T12	Delete from UI	confirm + DELETE + row removed
T13	Invalid marks	blocked before Service
T14	Login invalid	error, no navigation
T15	Login valid	Student screen opens
T16	Architecture audit	Controller imports no Hibernate/DAO

12. INTEGRATION GATE G1–G16

Gate	Yêu cầu	Evidence
G1	Toolchain	java/mvn/SceneBuilder/SQL
G2	LibraryDB	DB query
G3	Hibernate config	SessionFactory pass
G4	Mapping	3 tables
G5	DAO CRUD	source + SQL
G6	Repository	contract
G7	Service	use-case/validation
G8	Core smoke	console + DB
G9	JavaFX/FXML	screen load
G10	TableView	DB rows shown
G11	Selection	row -> form/id
G12	UI CRUD	create/update/delete
G13	Validation	invalid blocked
G14	Login	valid/invalid
G15	Break/Fix	>=3 failures diagnosed
G16	Explain	UI->Service->Repo->DAO->Hibernate->SQL

13. CHECKLIST SINH VIÊN TỰ ĐÁNH GIÁ

☐ 1. Tôi phân biệt SessionFactory và Session.

☐ 2. Tôi giải thích được vì sao transaction cần commit/rollback.

☐ 3. Tôi map được Student, Book và STUDENTS_BOOKS.

☐ 4. Tôi viết được DAO CRUD.

☐ 5. Tôi phân biệt DAO, Repository và Service.

☐ 6. Tôi chạy CRUD core trước khi gắn JavaFX.

☐ 7. Tôi hiểu fx:controller, fx:id, @FXML và initialize().

☐ 8. Tôi bind được TableView/TableColumn.

☐ 9. Tôi đồng bộ selected row với form và StudentID.

☐ 10. Tôi thực hiện Create/Update/Delete qua Service.

☐ 11. Tôi xử lý validation marks/input.

☐ 12. Tôi debug được FXMLLoadException và module opens.

☐ 13. Tôi hoàn thành Login page và navigation.

☐ 14. Tôi không gọi Hibernate Session từ JavaFX Controller.

☐ 15. Tôi có SQL/UI evidence cho các bước chính.

14. EVIDENCE PACKAGE PHẢI NỘP

Evidence	Nội dung
E1	Screenshot LibraryDB + 3 tables
E2	Project package tree
E3	Hibernate config không lộ credential thật
E4	DAO/Repository/Service source
E5	Console smoke CRUD
E6	Student Management UI
E7	Create before/after DB
E8	Update before/after DB
E9	Delete before/after DB
E10	Login invalid/valid screenshots
E11	Break-It diagnosis notes
E12	Self-check G1–G16

15. QUICK RUN CARD – DÙNG KHI DEBUG

```
1) SQL Server service running?
2) LibraryDB exists?
3) java -version = 21?
4) mvn compile/test pass?
5) hibernate.cfg.xml URL/user/password correct?
6) SessionFactory bootstrap?
7) STUDENTS / BOOKS / STUDENTS_BOOKS exist?
8) Core CRUD smoke test pass?
9) JavaFX dependencies/module correct?
10) FXML resource path correct?
11) fx:id / @FXML / onAction exact match?
12) Service injected before refresh?
13) TableView selection updates selectedStudentId?
14) UI CRUD verified in SQL Server?
15) Login valid/invalid verified?
```

16. ADVANCED EXTENSION & REFACTOR TRACK – TỰ NGHIÊN CỨU

ADVANCED: Các mục sau không thay đổi yêu cầu bắt buộc của Lab 07. Chỉ thực hiện sau khi G1–G16 PASS.

A1. Password Hashing

Thay plaintext password bằng BCrypt/Argon2-style adaptive hashing; login dùng matches().

A2. Association Entity Borrowing

Thay raw @ManyToMany bằng Borrowing khi cần borrowedDate, dueDate, status.

A3. Optimistic Locking
Bổ sung @Version cho Student/Book để xử lý concurrent updates.

A4. Transaction Boundary Refactor
Đưa transaction boundary lên Service cho multi-step use cases thay vì từng DAO tự commit.

A5. Dependency Injection / Controller Factory
Dùng FXMLLoader controllerFactory hoặc DI container thay manual/static service.

A6. JavaFX Background Task
Chạy DB work dài bằng Task/Service, tránh freeze JavaFX Application Thread.

A7. Search / Filter / Pagination
Thêm search Student và paging thay vì load toàn bộ table.

A8. Validation Model
Tách UI format validation và domain/business validation thành reusable validator.

A9. CSS Theme
Tách style sang CSS thay vì inline style trong FXML.

A10. Unit & Integration Tests
Service test bằng fake/mock repository; DAO integration test với DB test.

A11. Migration Tool
Dùng Flyway/Liquibase thay hbm2ddl=update trong hướng production.

A12. DTO/ViewModel
Không expose Entity trực tiếp cho UI ở application lớn; dùng form/view model.

A13. Audit Trail
Ghi createdAt/updatedAt và operation log cho CRUD.

A14. Login Session Context
Tạo CurrentUser/session context để giữ người đăng nhập mà không dùng static global tùy tiện.

A15. Packaging
Nghiên cứu jlink/jpackage để đóng gói JavaFX desktop app.

17. BÀI TẬP TỔNG HỢP NÂNG CAO

1. Refactor Student–Book thành Borrowing entity có borrowedDate/dueDate/status.
2. Bổ sung Book CRUD screen theo cùng Controller→Service→Repository→DAO.
3. Thêm chức năng Borrow/Return Book với transaction atomic.
4. Bổ sung search Student theo email/name và pagination.
5. Hash password và migrate dữ liệu test.
6. Chạy một DB operation giả lập chậm bằng JavaFX Task để chứng minh UI không freeze.
7. Viết ít nhất 5 automated tests cho Service/DAO.
8. Viết architecture note so sánh course manual Hibernate với framework-managed persistence.

18. SOURCE TRACEABILITY

- Lab 07 source – Building Desktop Application Using JavaFX and Hibernate: Introduction/Objectives, LibraryDB, STUDENTS/BOOKS/STUDENTS_BOOKS, Repository Architecture and CRUD.
- Activity 01 source: Gluon Scene Builder; design Manage Student and Login.
- Activity 02 source: Maven project, hibernate.cfg.xml, Book/Student POJO, StudentDAO CRUD, IStudentRepository/StudentRepository, IStudentService/StudentService, module-info.java.
- Activity 03 source: JavaFX project, architecture/module/reference, student-view.fxml, StudentController, show all students, row selection, CRUD and run.
- Activity 04 source: LoginGUI.fxml, LoginController and login success flow.